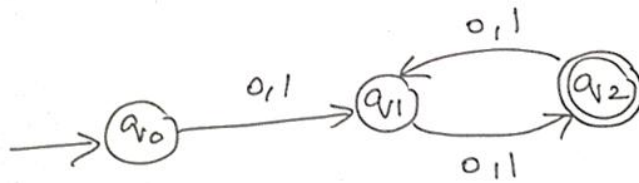


1.Design the regular expression for the language represented by the given automata



To find the regular expression for the language accepted by the given finite automaton.

Given Automata

- Initial State: q_0
- Intermediate State: q_1
- Final State: q_2
- Input Alphabet: $\Sigma = \{0,1\}$

Transitions

- $q_0 \rightarrow q_1$ on **0 or 1**
- $q_1 \rightarrow q_2$ on **0 or 1**
- $q_2 \rightarrow q_1$ on **0 or 1**

Procedure

Step 1: Analyze the Automata

The automaton starts from q_0 . It moves to q_1 by reading **0** or **1**. Again, by reading **0** or **1**, it reaches the final state q_2 .

After reaching q_2 , every two input symbols move the automaton from $q_2 \rightarrow q_1 \rightarrow q_2$. Hence, the automaton accepts strings containing an **even number of symbols**.

Step 2: Write the Regular Expression

Since each transition accepts either **0** or **1**, it is represented as:

$(0+1)$

Two transitions are needed to reach the final state:

$(0+1)(0+1)$

After reaching the final state, the same pair of symbols can repeat any number of times.

Therefore, the regular expression is:

$\{(0+1)(0+1)((0+1)(0+1))^*\}$

Language Accepted The automaton accepts **all binary strings of even length** (minimum length is 2).

$L = \{w \in \{0,1\}^* \mid |w| \text{ is even and } |w| \geq 2\}$

Final Answer

The required regular expression is:

$\{(0+1)(0+1)((0+1)(0+1))^*\}$

or equivalently,

$\{((0+1)(0+1))^{+}\}$

Conclusion

The given automaton accepts all binary strings with an **even number of symbols**. By analyzing the transitions, the corresponding regular expression is:

$\{(0+1)(0+1)((0+1)(0+1))^*\}$

This regular expression represents the same language accepted by the given automaton.

2. Consider the following grammar

$S \rightarrow aSc \mid T \mid U \mid \epsilon$

$T \rightarrow bTc \mid \epsilon$

$U \rightarrow aUb \mid \epsilon$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

Consider the following grammar:

$S \rightarrow aSc \mid T \mid U \mid \epsilon$

$T \rightarrow bTc \mid \epsilon$

$U \rightarrow aUb \mid \epsilon$

A) What is the language defined by the grammar?

Explanation

The grammar contains three productions starting from S.

1. $S \rightarrow aSc$
 - This production adds one 'a' at the beginning and one 'c' at the end.

- It generates strings with equal numbers of a's and c's.
- 2. $T \rightarrow bTc \mid \varepsilon$
 - This production generates strings with equal numbers of b's followed by c's.
 - The language is of the form $b^n c^n$, where $n \geq 0$.
- 3. $U \rightarrow aUb \mid \varepsilon$
 - This production generates strings with equal numbers of a's followed by b's.
 - The language is of the form $a^n b^n$, where $n \geq 0$.

The symbol ε represents the empty string.

Language Generated

The grammar generates the following types of strings:

- $a^n c^n$
- $b^n c^n$
- $a^n b^n$
- ε (empty string)

where $n \geq 0$.

B) Give the derivation tree for the word aaabbc.

Explanation

The given word is **aaabbc**.

According to the grammar:

- $S \rightarrow aSc$ always adds one 'a' and one 'c'.
- $T \rightarrow bTc$ generates equal numbers of b's and c's.
- $U \rightarrow aUb$ generates equal numbers of a's and b's.

The word **aaabbc** does not satisfy any of these conditions.

Therefore, the word **aaabbc cannot be generated from the given grammar.**

Hence, **a derivation tree cannot be constructed for this word.**

C) Is the grammar ambiguous? Justify.

Explanation

A grammar is said to be **ambiguous** if a string can be generated in more than one way, resulting in two or more different parse trees or derivations.

In the given grammar:

- Each valid string has only one possible derivation.
- No string can be derived in two different ways.

Conclusion

The given grammar generates strings of the form $a^n c^n$, $b^n c^n$, $a^n b^n$, and the empty string (ϵ). The word **aaabbc** cannot be generated by this grammar, so a derivation tree cannot be drawn. Since every valid string has a unique derivation, the grammar is **not ambiguous**.

3. Design a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression
 $00^*1 + 11^*0$

Regular Expression:

$00^*1 + 11^*0$

Aim

To design a Non-Deterministic Finite Automaton (NFA) with ϵ -transitions for the given regular expression **$00^*1 + 11^*0$** .

Theory

A **Non-Deterministic Finite Automaton (NFA)** is a finite automaton in which a state can have one or more transitions for the same input symbol. An **ϵ -transition** (epsilon transition) allows the automaton to move from one state to another without reading any input symbol.

The given regular expression is:

$00^*1 + 11^*0$

Here, '+' represents the **union (OR)** operation.

The expression consists of two parts:

- **00^*1**
- **11^*0**

The NFA is constructed by creating separate paths for both expressions and connecting them with ϵ -transitions from a common start state.

Procedure

Step 1: Analyze the Regular Expression

Expression 1: 00^*1

- Begins with one **0**
- Followed by zero or more **0's**
- Ends with one **1**

Examples: 01, 001, 0001, 00001

Expression 2: 11^*0

- Begins with one **1**
- Followed by zero or more **1**'s
- Ends with one **0**

Examples: 10, 110, 1110, 11110

Step 2: Construct the NFA

1. Create the start state **q0**.
2. From **q0**, draw two ϵ -transitions.
3. One branch represents **00*1**.
4. The other branch represents **11*0**.
5. The automaton accepts the string if it reaches either **q3** or **q7**.

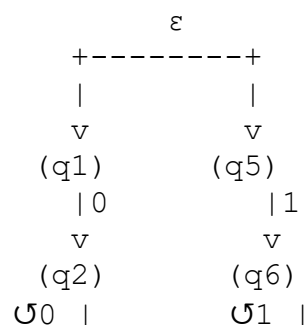
Transition Table

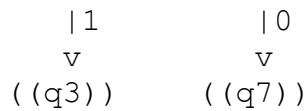
Present State	Input	Next State
q0	ϵ	q1
q0	ϵ	q5
q1	0	q2
q2	0	q2
q2	1	q3
q5	1	q6
q6	1	q6
q6	0	q7

Start State: q0

Final States: q3, q7

NFA Diagram





q0 is the Start State
q3 and q7 are Final States

Working of the NFA

- The automaton starts from **q0**.
- It takes an **ϵ -transition** to either **q1** or **q5**.
- The first branch accepts strings of the form **00*1**.
- The second branch accepts strings of the form **11*0**.
- If the automaton reaches **q3** or **q7**, the input string is accepted.

Language Accepted

The NFA accepts all strings represented by the regular expression:

$$L = \{001\} \cup \{110\}$$

Examples of accepted strings are:

- 01
- 001
- 0001
- 10
- 110
- 1110

Final Answer

The required **NFA with ϵ -transitions** for the regular expression **001 + 110** is constructed by connecting two separate NFAs using ϵ -transitions from a common start state.

- **Start State:** q0
- **Final States:** q3 and q7

Conclusion

The given regular expression **001 + 110** is successfully represented using a Non-Deterministic Finite Automaton (NFA) with ϵ -transitions. The ϵ -transitions allow the automaton to choose either branch without consuming any input symbol. One branch accepts strings of the form **00*1**, and the other accepts strings of the form **11*0**. Therefore, the constructed NFA correctly represents the given regular expression.

4. i) Show that if L_1 and L_2 are regular then $(L_1 + L_2)$ also regular
ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

Theory

A **regular language** is a language that can be accepted by a **Finite Automaton (FA)** or represented by a **Regular Expression**.

Let,

- L_1 be a regular language.
- L_2 be another regular language.

Since both languages are regular, there exist finite automata M_1 and M_2 that accept L_1 and L_2 respectively.

The operator '+' denotes the **union** of two languages.

Therefore,

$$L = L_1 + L_2 = L_1 \cup L_2$$

The union language contains all the strings that belong to either L_1 or L_2 .

Proof

Since L_1 and L_2 are regular languages, there are finite automata for both languages.

To construct an automaton for $L_1 \cup L_2$:

1. Create a new start state called q_0 .
2. Draw ϵ -transitions from q_0 to the start states of automata M_1 and M_2 .
3. Keep all transitions and states of M_1 and M_2 unchanged.
4. The accepting states of the new automaton are all the accepting states of M_1 and M_2 .

If a string is accepted by M_1 or by M_2 , then it is accepted by the new automaton.

Hence, the new automaton accepts $L_1 \cup L_2$.

Therefore,

$L_1 + L_2$ is also a regular language.

Result

Since a finite automaton can be constructed for the union of two regular languages, **the class of regular languages is closed under the union operation.**

Hence,

If L_1 and L_2 are regular, then $(L_1 + L_2)$ is also regular.

Conclusion

Regular languages remain regular even after applying the **union operation**. Therefore, the union of any two regular languages is also a regular language.

(ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify.

Theory

The language consists of strings containing only the symbol 'a', where the number of a's is a **prime number**.

Examples:

- aa (length = 2)
- aaa (length = 3)
- aaaaa (length = 5)
- aaaaaaa (length = 7)
- aaaaaaaaaa (length = 11)

Only strings whose lengths are prime numbers belong to the language.

Justification Using Pumping Lemma

Assume that the language $L = \{a^p \mid p \text{ is prime}\}$ is regular.

According to the **Pumping Lemma**, every sufficiently long string in a regular language can be divided into three parts xyz such that:

- $|xy| \leq n$
- $|y| > 0$
- xy^iz belongs to the language for every $i \geq 0$

Choose a string $w = a^p$, where p is a sufficiently large prime number.

Since $|y| > 0$, pumping the substring y (that is, repeating it one or more times) increases the total number of a's.

The length of the new string becomes different from p .

This new length will not always be a prime number.

Therefore, the pumped string does not belong to the language.

This violates the Pumping Lemma.

Hence, our assumption that the language is regular is false.

Result

The language $L = \{a^p \mid p \text{ is prime}\}$ does not satisfy the Pumping Lemma.

Therefore, it cannot be accepted by any finite automaton.

Hence, the language is **not regular**.

Final Answer

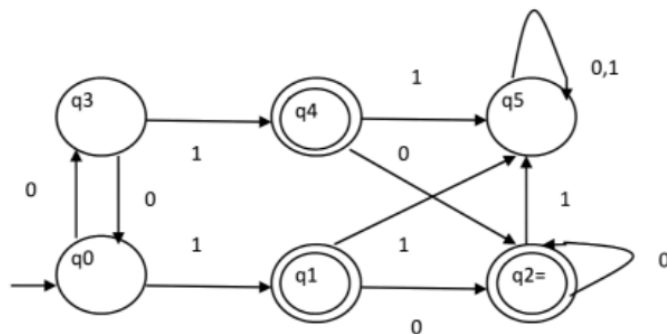
(i) If L_1 and L_2 are regular languages, then $L_1 + L_2$ ($L_1 \cup L_2$) is also a regular language because regular languages are **closed under the union operation**.

(ii) The language $\{a^p \mid p \text{ is prime}\}$ is **not a regular language** because it does not satisfy the **Pumping Lemma**.

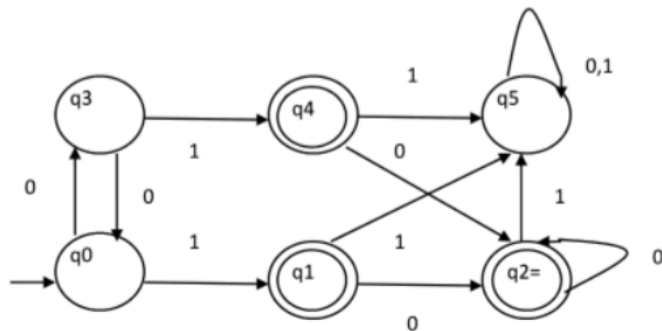
Conclusion

The union of two regular languages is always regular because a finite automaton can be constructed to accept the union of both languages. However, the language $\{a^p \mid p \text{ is prime}\}$ is **not regular** because a finite automaton cannot determine whether the length of a string is a prime number, and the language violates the Pumping Lemma. This proves that the given language is not regular.

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



DFA Minimization

1. Given DFA

- **States:** $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
- **Alphabet:** $\Sigma = \{0,1\}$
- **Initial State:** q_0
- **Final States:** $\{q_1, q_2, q_4\}$

2. Transition Table

Present State 0 1

$\rightarrow q_0$	$q_3 \ q_1$
q_3	$q_0 \ q_4$
$*q_1$	$q_2 \ q_5$
$*q_2$	$q_2 \ q_5$
$*q_4$	$q_2 \ q_5$
q_5	$q_5 \ q_5$

3. Initial Partition

Separate final and non-final states:

$$P_0 = \{\{q_1, q_2, q_4\}, \{q_0, q_3, q_5\}\}$$

4. Equivalent States

- q_1, q_2 and q_4 have the same transitions:
 - On 0 $\rightarrow$ **Final state**
 - On 1 $\rightarrow$ **Non-final state**

Therefore,

$$q_1 \equiv q_2 \equiv q_4$$

- **q_0 and q_3** also have identical behaviour:
 - On **0** → **Non-final state**
 - On **1** → **Final state**

Therefore,

$$q_0 \equiv q_3$$

- **q_5** is a dead state (self-loop on both inputs), so it remains separate.

5. Final Equivalence Classes

$$P = \{\{q_0, q_3\}, \{q_1, q_2, q_4\}, \{q_5\}\}$$

Rename the states:

- **A** = $\{q_0, q_3\}$ (Initial)
- **B** = $\{q_1, q_2, q_4\}$ (Final)
- **C** = $\{q_5\}$ (Dead state)

6. Minimized DFA Transition Table

State 0 1

→A A B

*B B C

C C C

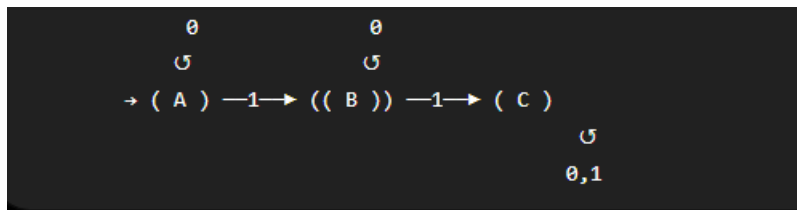
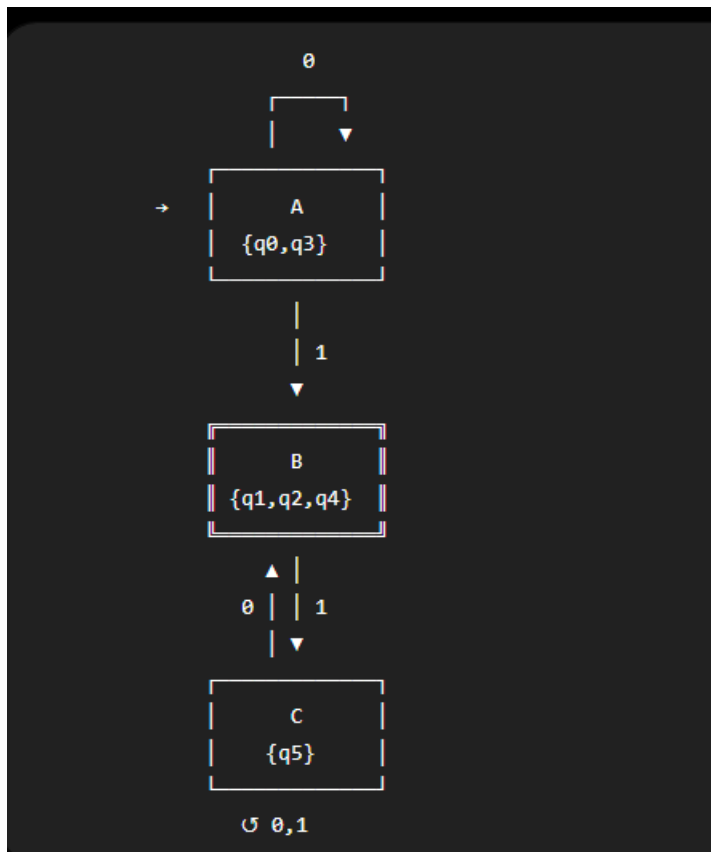
7. Minimized DFA

- **States:** {A, B, C}
- **Initial State:** A
- **Final State:** B

Transitions:

- **A** → 0 → **A**, → 1 → **B**
- **B** → 0 → **B**, → 1 → **C**
- **C** → 0 → **C**, → 1 → **C**

Result: The original DFA with **6 states** is minimized to **3 states: A, B, and C**.



Here:

$$\begin{aligned}
 A &= \{q_0, q_3\} \\
 B &= \{q_1, q_2, q_4\} \\
 C &= \{q_5\}
 \end{aligned}$$